

SQL - II

Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



SQL-I Concepts

- Perform computations over sets of values in multiple rows
- Basic aggregation functions supported by every SQL system are
  - **Minimum,**
  - **Maximum,**
  - **Sum,**
  - **Average,**
  - **Count.**

```
Select A1,A2,…,An
From   R1,R2, …,Rm
Where  condition
Group By columns
Having condition
```

"Aggregation" functions over values in multiple rows: min, max, sum, avg, count

New clauses

- Group by allows us to partition our relations into groups and then will compute aggregated aggregate functions over each group independently.
- Having condition allows us to test filters on the results of aggregate values.
- Having condition will apply to the groups that we generate from the group by clause.

SQL-II

- Types of **SQL-Join**
- **Sub-queries** in
  - **Select**
  - **From**
  - **Where**
- Etc.

SQL: Cartesian product with condition

```
Select A1,A2,…,An
From   R1,R2, …,Rm
Where  condition
```

Cartesian product (R1XR2…) with condition

```
select distinct sName, major
from Student, Apply
where Student.sID = Apply.sID;
```

SQL: The JOIN family of operators

`Natural Join` ⋈ MySQL, ~~SQL-Server~~

`Inner Join On Condition`

`Inner Join Using(`*attrs*`)`

`Left | Right | Full Outer Join`

Natural JOIN & Inner JOIN operators

- Joins are used to combine rows from two or more tables based on a related column between them
- Joins merge data from multiple tables based on common columns (usually foreign keys)
- **NATURAL JOIN**
  - Automatically identifies **columns with identical names** in both tables
- **INNER JOIN:**
  - When you need only matching records

MySQL: Natural Join

```
select *
from table1 natural join table2
```

	C1	C2
1	1	A
2	1	B
3	2	C
4	*NULL*	D
5	3	E
6	7	DA

table1

	C1	C3
1	1	XA
2	2	MB
3	2	NX
4	*NULL*	MO
5	4	XY
6	5	TF

table2

C1	C2	C3
1	A	XA
1	B	XA
2	C	MB
2	C	NX

- MySQL **automatically** identifies **columns with identical names** in both tables
- It joins the tables using **equality conditions on those matching columns**
- The result includes each matching column only once (**no duplicated attributes**)
- Can be **risky** because they rely on column names, which might change over time !

Example: College Admission Database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

	cName	state	enrollment
1	Stanford	CA	15000
2	Berkeley	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

	sID	sName	GPA	sizeHS
1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800
8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

	sID	cName	major	decision
1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

MySQL: Natural Join

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

```
/*************************************************************
  MySQL Natural JOIN
  Student names and majors for which they've applied
*************************************************************/

select  distinct sName, major
from Student natural join Apply;
```

SQL-Server not support Natural Join

```
/*************************************************************
  NATURAL JOIN
  Student names and majors for which they've applied
*************************************************************/

select distinct sName, major
from Student natural join Apply;


select distinct sName, major
from Student, Apply
where Student.sID = Apply.sID;
```

SQL: Inner Join

When you need only matching records

```
select *
from table1 inner join table2
on table1.C1 = table2.C1
```

	C1	C2
1	1	A
2	1	B
3	2	C
4	*NULL*	D
5	3	E
6	7	DA

table1

	C1	C3
1	1	XA
2	2	MB
3	2	NX
4	*NULL*	MO
5	4	XY
6	5	TF

table2

	C1	C2	C1	C3
1	1	A	1	XA
2	1	B	1	XA
3	2	C	2	MB
4	2	C	2	NX

MySQL: Inner Join

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

```
select  distinct sName, major
from Student natural join Apply;

/*** Rewrite using INNER JOIN ***/

select  distinct sName, major
from Student inner join Apply
on Student.sid = Apply.sid;

/*** Abbreviation is JOIN ***/

select  distinct sName, major
from Student join Apply
on Student.sid = Apply.sid;
```

Cartesian product with condition VS Inner Join

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

```
/*** Cartesian product with condition ***/

select distinct sName, major
from Student, Apply
where Student.sID = Apply.sID; /** WHERE filters rows **/
```

```
/*** INNER JOIN ***/

select  distinct sName, major
from Student join Apply
on Student.sid = Apply.sid; /** ON connects tables **/
```

SQL: Inner Join

- Student names and majors for which they've applied with condition

```
select distinct sname, major
from Student join Apply
on Student.sid = Apply.sid
where sizeHS < 1000 and major = 'CS' and cname = 'Stanford'
```

SQL: Inner Join with 3 Tables
Some DBMS not support 3-ways Join

```
/*************************************************************
  INNER JOIN
  Student names and majors for which they've applied
*************************************************************/
select Apply.sid, sName, gpa, Apply.cname, enrollment
from Apply join Student join College
on Student.sID = Apply.sID and Apply.cname = College.cname

Some DBMS not support 3-way joins

/*** Rewrite using binary INNER JOIN ***/
select Apply.sid, sName, gpa, Apply.cname, enrollment
from (Apply join Student on Student.sID = Apply.sID)
join College on Apply.cname = College.cname
```

SQL: OUTER JOIN operators

- **INNER JOIN:**
  - When you need only matching records
- **OUTYER JOIN:**
  - When you need both matching records & non matching records

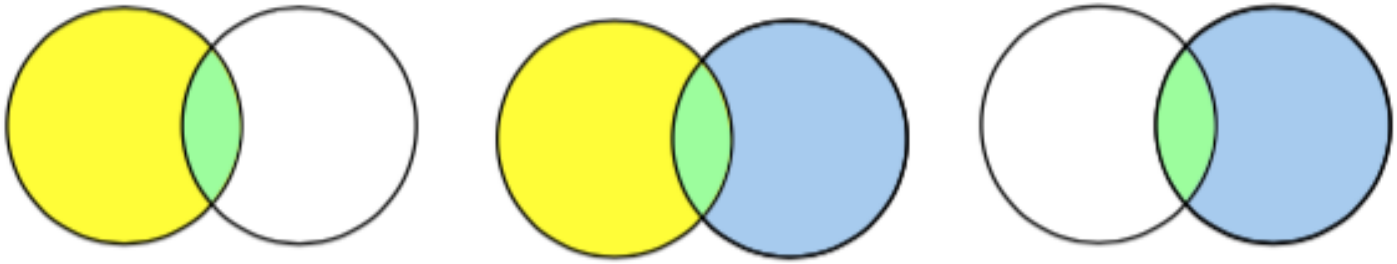
- **LEFT OUTER JOIN:**
  - Entire left table + intersection
  - All left table records
- **RIGHT OUTER JOIN:**
  - Entire right table + intersection
  - All right table records
- **FULL OUTER JOIN:**
  - All records from both tables

SQL: Outer JOIN

Outer Joins

You can retrieve both non-matching and matching rows using an outer join.

Outer joins include left, full, and right outer joins. Many tables can be referenced in outer joins. The tables are processed two tables at a time.



Left Full Right

SQL: Left Outer Join

```
select *
from
table1 left join table2
on table1.C1 = table2.C1
```

	C1	C2
1	1	A
2	1	B
3	2	C
4	*NULL*	D
5	3	E
6	7	DA

table1

	C1	C3
1	1	XA
2	2	MB
3	2	NX
4	*NULL*	MO
5	4	XY
6	5	TF

table2

	C1	C2	C1	C3
1	1	A	1	XA
2	1	B	1	XA
3	2	C	2	MB
4	2	C	2	NX
5	*NULL*	D	*NULL*	*NULL*
6	3	E	*NULL*	*NULL*
7	7	DA	*NULL*	*NULL*

SQL: Right Outer Join

```
select *
from
table1 right join table2
on table1.C1 = table2.C1
```

	C1	C2
1	1	A
2	1	B
3	2	C
4	*NULL*	D
5	3	E
6	7	DA

table1

	C1	C3
1	1	XA
2	2	MB
3	2	NX
4	*NULL*	MO
5	4	XY
6	5	TF

table2

	C1	C2	C1	C3
1	1	A	1	XA
2	1	B	1	XA
3	2	C	2	MB
4	2	C	2	NX
5	*NULL*	*NULL*	*NULL*	MO
6	*NULL*	*NULL*	4	XY
7	*NULL*	*NULL*	5	TF

SQL: Full Outer Join

```
select *
from
table1 full join table2
on table1.C1 = table2.C1
```

	C1	C2	C1	C3
1	1	A	1	XA
2	1	B	1	XA
3	2	C	2	MB
4	2	C	2	NX
5	*NULL*	D	*NULL*	*NULL*
6	3	E	*NULL*	*NULL*
7	7	DA	*NULL*	*NULL*
8	*NULL*	*NULL*	*NULL*	MO
9	*NULL*	*NULL*	4	XY
10	*NULL*	*NULL*	5	TF

	C1	C2
1	1	A
2	1	B
3	2	C
4	*NULL*	D
5	3	E
6	7	DA

table1

	C1	C3
1	1	XA
2	2	MB
3	2	NX
4	*NULL*	MO
5	4	XY
6	5	TF

table2

SQL: Inner Join

- Student application info: name, ID, college name, major

- Select sName, Student.sID, cName, major
- from Student inner join Apply on Student.sID = Apply.sID;

	sName	sID	cName	major
1	Amy	123	Berkeley	CS
2	Amy	123	Cornell	EE
3	Amy	123	Stanford	CS
4	Amy	123	Stanford	EE
5	Bob	234	Berkeley	biology
6	Craig	345	Cornell	bioengineering
7	Craig	345	Cornell	CS
8	Craig	345	Cornell	EE

SQL: Left Outer Join (Left Join)

```
-- Student application info: name, ID, college name, major

 -- INNER JOIN
select sName, Student.sID, cName, major
from Student inner join Apply on Student.sID = Apply.sID;

-- Include students who haven't applied anywhere
select sName, Student.sID, cName, major
from Student left outer join Apply on Student.sID = Apply.sID;

-- Abbreviation is LEFT JOIN
select sName, Student.sID, cName, major
from Student left join Apply on Student.sID = Apply.sID;
```

SQL: Left Join

```
/*** Include students who haven't applied anywhere ***/

select sName, Student.sID, cName, major
from Student left join Apply on Student.sID = Apply.sID;
```

	sName	sID	cName	major
1	Amy	123	Berkeley	CS
2	Amy	123	Cornell	EE
3	Amy	123	Stanford	CS
4	Amy	123	Stanford	EE
5	Bob	234	Berkeley	biology
6	Craig	345	Cornell	bioengineering
7	Craig	345	Cornell	CS
8	Craig	345	Cornell	EE
9	Craig	345	MIT	bioengineering
10	Doris	456	NULL	NULL
11	Craig	543	MIT	CS
12	Edward	567	NULL	NULL
13	Amy	654	NULL	NULL

SQL: Right Outer Join (Right Join)

```
insert into Apply values (322, 'MIT', 'history', 'N');
insert into Apply values (322, 'MIT', 'psychology', 'Y');
/*** 322 is a student not exist in student table ***/
```

```
/*** and applications without matching students *
select sName, Student.sID, cName, major
from Student right join Apply on Student.sID =
```

Results | Messages

	sName	sID	cName	major
1	Amy	123	Berkeley	CS
2	Amy	123	Cornell	EE
3	Amy	123	Stanford	CS
4	Amy	123	Stanford	EE
5	Bob	234	Berkeley	biology
6	NULL	NULL	MIT	history
7	NULL	NULL	MIT	psychology
8	Craig	543	MIT	CS
9	Fay	678	Stanford	history
10	Jay	765	Cornell	history
11	Jay	765	Cornell	psychology
12	Jay	765	Stanford	history
13	Helen	987	Berkeley	CS
14	Helen	987	Stanford	CS

SQL: Full Outer Join (Full Join)

```
/*****************************************************
  FULL OUTER JOIN
  Student application info
*****************************************************

/*** Include students who haven't applied
anywhere ***/
/*** and applications without matching
students ***/

select sName, Student.sID, cName, major
from Student full outer join Apply on
Student.sID = Apply.sID;
```

	sName	sID	cName	major
1	Amy	123	Berkeley	CS
2	Amy	123	Cornell	EE
3	Amy	123	Stanford	CS
4	Amy	123	Stanford	EE
5	Bob	234	Berkeley	biology
6	Lori	321	NULL	NULL
7	NULL	NULL	MIT	history
8	NULL	NULL	MIT	psychology
9	Kevin	432	NULL	NULL
10	Doris	456	NULL	NULL
11	Craig	543	MIT	CS
12	Edward	567	NULL	NULL
13	Amy	654	NULL	NULL
14	Fay	678	Stanford	history
15	Jay	765	Cornell	history
16	Jay	765	Cornell	psychology
17	Jay	765	Stanford	history
18	Gary	789	NULL	NULL
19	Helen	987	Berkeley	CS
20	Helen	987	Stanford	CS

What is a Subquery (Inner Query)?

Definition: A subquery is a SELECT statement that is **embedded (nested) inside another SQL statement**.

Key Characteristics:

- Also called "nested query" or "inner query"
- Always enclosed in parentheses ( )
- Executes before the **outer query** (usually)
- **Returns data that the outer query uses**
- Can be nested multiple levels deep

Basic Structure:

```
SELECT columns          -- Outer Query
FROM table
WHERE column operator ( SELECT column    -- Subquery (Inner Query)
                        FROM table
                        WHERE condition);
```

Simple Subquery Example

Problem: Find all students with GPA higher than Fay's GPA

Without Subquery (2 steps):

- *-- Step 1: Find Fay's GPA*
- SELECT distinct GPA FROM Student WHERE sName = 'Fay'; *-- Result: 3.8*
- *-- Step 2: Use that value*
- SELECT sName, GPA FROM Student WHERE GPA > 3.8;

With Subquery (1 step):

- SELECT sName, GPA
- FROM Student
- WHERE GPA > (**SELECT distinct GPA FROM Student WHERE sName = 'Fay'**);

Advantage: Dynamic, single query, no manual intervention needed

Subquery Execution Process

Problem: Find all students with GPA higher than Fay's GPA

- SELECT sName, GPA
- FROM Student
- WHERE GPA > (SELECT distinct GPA FROM Student WHERE sName = ‘Fay’);

Step-by-Step Execution:

Inner Query Executes First

- SELECT distinct GPA FROM Student WHERE sName = ‘Fay’;
- 3.8

Result Substituted in Outer Query

- SELECT sName, GPA
- FROM Student
- WHERE GPA > 3.8;

SQL: Sub-query in Where

- **Purpose:** Filter rows based on conditions
- **Operators:**
  - **Single value (**Scalar Subquery**):** =, >, <, >=, <=, <>
  - **Multiple values (**Multi-row Subquery**):** IN, NOT IN, ANY, ALL
  - **Existence check (**EXISTS Subquery**):** EXISTS, NOT EXISTS

```
/*************************************************************
  IDs and names of students applying to CS
*************************************************************/
select sID, sName
from Student
where sID in (select sID from Apply where major = 'CS');
```

Sub-query in Where: Simulation of Left Join

```
-- Student application info: name, ID, college name, major
 -- INNER JOIN
select sName, Student.sID, cName, major
from Student inner join Apply on Student.sID = Apply.sID;

-- Include students who haven't applied anywhere
-- LEFT JOIN
select sName, Student.sID, cName, major
from Student left join Apply on Student.sID = Apply.sID;

-- Sub-Query
select sName, Student.sID, cName, major
from Student, Apply
where Student.sID = Apply.sID
union
select sName, sID, NULL, NULL
from Student
where sID not in (select sID from Apply);
```

SQL: Sub-query in Where

- **Purpose:** Filter rows based on conditions
- **Operators:**
  - **Single value (**Scalar Subquery**):** =, >, <, >=, <=, <>
  - **Multiple values (**Multi-row Subquery**):** IN, NOT IN, ANY, ALL
  - **Existence check (**EXISTS Subquery**):** EXISTS, NOT EXISTS

```
/*****************************************************
  Average GPA of CS applicants
*****************************************************/
select AVG(GPA)
from Student
where sID in (select sID from Apply where major = 'CS');
/*****************************************************
 without 'In', this alternative query is incorrect !!!
*****************************************************/
select AVG(GPA)
from Student, Apply
where Student.sID = Apply.sID and major = 'CS';
```

SQL: Sub-query in Where

- **Purpose:** Filter rows based on conditions
- **Operators:**
  - **Single value (**Scalar Subquery**):** =, >, <, >=, <=, <>
  - **Multiple values (**Multi-row Subquery**):** IN, NOT IN, ANY, ALL
  - **Existence check (**EXISTS Subquery**):** EXISTS, NOT EXISTS
- Use of EXISTS, NOT EXISTS
- What ever subquery returns => **Having row or not?**
- Biggest college

```
select cName
from College C1
where not exists (select * from College C2
                  where C2.enrollment > C1.enrollment);
/Student with highest GPA  ***/
select sName
from Student C1
where not exists (select * from Student C2
                  where C2.GPA > C1.GPA);
```